Inheritance in C++

🔹 Definition:

Inheritance is a mechanism in C++ where one class (derived or child class) inherits the properties (data members and member functions) of another class (base or parent class). It supports the concept of reusability and establishes a relationship between classes.

🔹 Need:

- To promote **code reusability** by allowing derived classes to reuse code from base classes.
- To represent **hierarchical relationships** naturally (e.g., Animal → Mammal → Dog).
- To implement **polymorphism**, enabling flexibility and maintainability.

🔹 Use Cases:

- Classifying systems like Employee types (Manager, Developer).
- GUI frameworks where Button inherits Widget.
- Financial systems (Account → SavingsAccount, CurrentAccount).

🔹 Syntax:

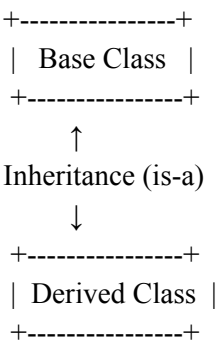
cpp
CopyEdit

```
class Base {
  // base class members
};

class Derived : access_specifier Base {
  // additional members
};
```

🔹 Vein Diagram:

pgsql
CopyEdit



What are Child and Parent Classes?

You may have heard the terms like child class or derived class and parent class, or the base class, a lot of times in inheritance; let's understand these terms in detail:

- Child Classes:
  A child class is a class that inherits the properties from another class, also known as the derived class. The child class can access the data members and the member functions of the parent class according to the visibility mode specified during the declaration of the child class.
- Parent Classes:
  The parent class is the class from which the child is being derived and inherited; it is also known as the base class. Depending on the type of inheritance being applied, a single-parent class can derive multiple child classes, and multiple-parent classes can inherit a single-child class.

```
class A
{ // Code

};
class B: public A
{
// Code
};
```

Here, class B is inherited from class A, then class B will behave as the child class, and class A will act like a parent or base class.

🌿 Topic: Defining Derived Class

🔹 **Syntax:**

```
cpp
CopyEdit
class Base {
public:
   int x;
   void show() { cout << "Base"; }
};

class Derived : public Base {
public:
   void display() { cout << x; }
};
```

- **Example 1 (Beginner):**

cpp
CopyEdit

```
#include<iostream>
using namespace std;

class Animal {
public:
    void sound() {
        cout << "Animals make sound\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Dog barks\n";
    }
};

int main() {
    Dog d;
    d.sound();
    d.bark();
    return 0;
}
```

- **Example 2 (Beginner):**

cpp
CopyEdit

```
class Parent {
public:
    void greet() { cout << "Hello from Parent\n"; }
};

class Child : public Parent {
public:
    void welcome() { cout << "Welcome from Child\n"; }
};

int main() {
    Child c;
    c.greet();
```

```
    c.welcome();
}
```

🔹 **Example 3 (Medium):**

cpp
CopyEdit

```
class Employee {
protected:
    string name;
    int id;
public:
    void setDetails(string n, int i) {
        name = n; id = i;
    }
};

class Manager : public Employee {
    float bonus;
public:
    void setBonus(float b) { bonus = b; }
    void display() {
        cout << "Name: " << name << ", ID: " << id << ", Bonus: " << bonus << endl;
    }
};

int main() {
    Manager m;
    m.setDetails("Ravi", 101);
    m.setBonus(1200.5);
    m.display();
}
```

🌿 Topic: Modes of Inheritance

🔹 **Types:**

- **Public**: Public & protected members remain accessible in derived class.
- **Protected**: Public & protected members become protected.
- **Private**: All inherited members become private.

🔹 **Syntax:**

cpp

CopyEdit
class Derived : public/protected/private Base {};

Visibility Modes

An important feature in inheritance is to know how the derived class will inherit the base class and which members of the base class will be acquired and accessible by the derived class. For this, we have three types of visibility modes (also known as access modifiers) for all types of Inheritance in C++:

- Public visibility mode:
  In public visibility mode, when we inherit a child class from the parent class, then the public, private, and protected members of the base class remain public, private, and protected members, respectively, in the derived class. The public visibility mode retains the accessibility of all the parent class members.
  The public members of the parent are accessible by the child's class and all other classes. The protected members of the base class are accessible only inside the derived class and its inherited classes. However, the private members are not accessible to the child class.

The syntax of using visibility modes is as follows:

```
class child_class_name :: visibility_mode parent_class_name
{

}
```

The visibility_mode can be public, private, or protected.

Let's see an example to understand the concept of public visibility mode:

```
class Parent
{
   private:
    int x1;

   protected:
    int x2;

   public:
    int x3;
};
class Child: public Parent
{

};
```

The protected variable x2 will be inherited from the Parent class and accessible in the Child class. Similarly, the public variable x3 will be inherited from the Parent class and accessible in the Child class, but the private variable x1 will not be accessible in the Child class.

- Private Visibility Mode:
  In the private visibility mode, when we inherit a child class from the parent class, all the base class members (public, private and protected members) will become private in the derived class.
  The access of these members outside the derived class is restricted, and the member functions of the derived class can only access them.

Let's see an example to understand the concept of private visibility mode:

```
class Parent
{
   private:
    int x1;

   protected:
    int x2;

   public:
    int x3;
};
class Child: private Parent
{

};
```

As the visibility mode is private, so, all the members of the base class have become private in the derived class. The error is thrown when the object of the derived class tries to access these members outside the derived class.

- Protected Visibility Mode:
  In the protected visibility mode, when we inherit a child class from the parent class, all the base class members will become the derived class's protected members.
  Because of this, these members are now only accessible by the derived class and its member functions. These members can also be inherited and accessible to the inherited subclasses of this derived class.

Let's see an example to understand the concept of protected visibility mode:

```
class Parent
```

```
{
   private:
    int x1;

   protected:
    int x2;

   public:
    int x3;
};
class Child: protected Parent
{

};
```

As the visibility mode is protected, the protected and the public members of the Parent class become the protected members of the Child class.

The protected variable x2 will be inherited from the Parent class and accessible in the Child class. Similarly, the public variable x3 will be inherited from the parent class as the protected member and will be accessible in the Child class, but the private variable x1 will not be accessible in the Child class.

Given table will summarize the above three modes of inheritance in C++:

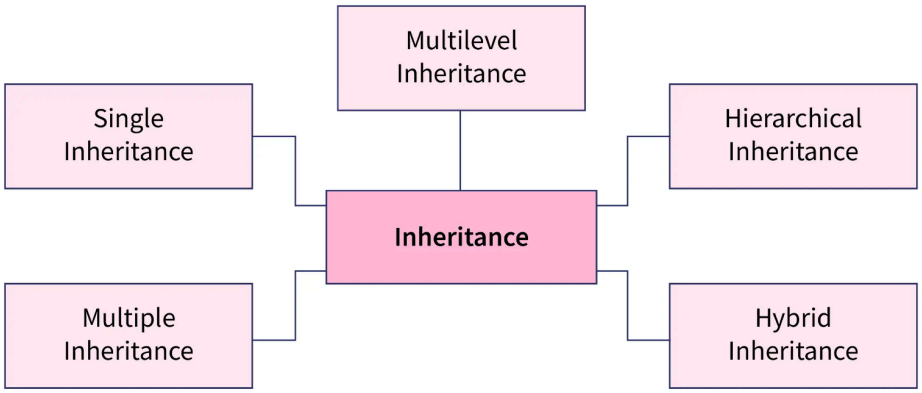
	Derived Class	Derived Class	Derived Class
Base Class	**Private Mode**	**Protected Mode**	**Public Mode**
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

SCALER Topics

NOTE: If no visibility mode is specified, then, by default, the private mode is considered.

Depending on the way the child class is derived from the parent class or how many parent classes a child class inherits, we have the following types of inheritance in C++, namely:

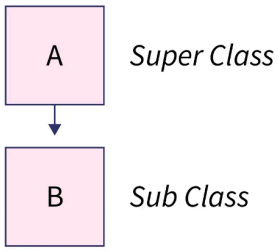
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance



SCALER Topics

Single Inheritance in C++:

Single inheritance is one of the simplest inheritance among other types of inheritance in C++, in which the child class is derived only from the single parent class.



SCALER Topics

As you can see from the image above, class B is derived from a single-parent class A.

Let's see the syntax to perform single inheritance in C++:

```
class A
{


};
class B: visibility_mode A
{


}
```

Here, class A is the parent class, class B is the child class, and class B is inherited from class A.

Let's see the example of single inheritance:

```
#include <bits/stdc++.h>
using namespace std;
// base class

class Animal

{

public:

   // constructor of the base class

   Animal()

   {

      cout << "I am an animal.\n";

   }

};

 // derived class
```

```
class Dog: public Animal

{

public:

   // constructor of the derived class

   Dog()

   {

      cout << "I am a dog.\n";

   }

};
int main()
{
   // create an object of the child's class
   Dog obj;
   return 0;
}
```

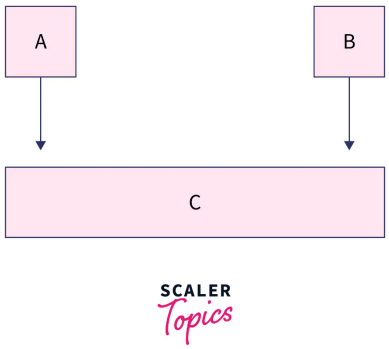
The output of the above example is:

```
I am an animal.
I am a dog.`
```

In the above example, the child class Dog inherits the parent class Animal in a public visibility mode. So, all the public and protected member functions and data members of the Animal class are directly accessible in the child class Dog.

Multiple Inheritance in C++

Multiple inheritances are another type of inheritance in c++ in which the child class inherits its properties from more than one base class. This means the child class is inherited from multiple parent classes, so the child class can derive the combined features of all these classes. The child class accesses all base class data members according to the visibility mode used.



As you can see from the image above, class C is derived from both the base classes A and B.

Let's see the syntax to perform multiple inheritances in cpp:

```
class A
{


};
class B
{


};
class C: visibility_mode_1 A, visibility_mode_2 B
{

};
```

Here, class A and class B are the parent classes, and class C is the child class inherited from both parent classes A and B.

Let's see the example of multiple inheritances:

```
#include <bits/stdc++.h>
using namespace std;

// base class

class Animal

{
```

```
public:
    // constructor of the base class
    Animal()
    {
        cout << "I am an animal.\n";
    }
};
class Speak
{
public:
    // constructor of the base class
    Speak()
    {
        cout << "I can speak.\n";
    }
};
 // derived class
class Dog: public Animal, public Speak
{
};
```

```
int main()
{
   // create an object of the child's class
   Dog obj;
   return 0;
}
```

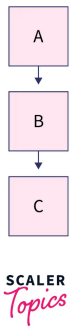
Output:

```
I am an animal.
I can speak.
```

In the above example, the child class Dog inherits the parent classes Animal and Speak in a public visibility mode. So, all the public and protected member functions and data members of the base classes are directly accessible in the child class. Also, when you create an object for a derived class, the constructors for the two base class, Animal and Speak, are called.

Multilevel Inheritance in C++:

Multilevel inheritance is one type of inheritance in C++ in which a derived class inherits all its properties from a class that itself inherits from another class, i.e., the parent class for one is a child class for the other.



As you can see from the image above, class C is derived from class B, which in turn is derived from class A.

Let's see the syntax to perform multilevel inheritance in cpp:

```
class A
{
```

```
};
class B: visibility_mode A
{


};
class C: visibility_mode B
{

};
```

Here, class A is the parent class, class B is the child class, which is inherited from the parent class A, and class B is the parent class for class C, which is being inherited from class B.

Let's see the example of multilevel inheritance:

```
#include <bits/stdc++.h>
using namespace std;

class Animal

{

public:

   // constructor of the base class

   Animal()

   {

      cout << "I am an animal.\n";

   }

};


class Dog: public Animal

{
```

```
public:

    // constructor of the derived class

    Dog()

    {

        cout << "I am a dog.\n";

    }

};
class Pug: public Dog

{

public:

    Pug()

    {

        cout << "I am a pug.\n";

    }

};


int main()
{
    // create an object of the child's class
    Pug obj;
    return 0;
}
```

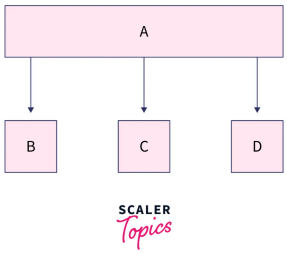
The output of the above example is:

I am an animal.
I am a dog.
I am a pug.

Here, in the example, the object of the derived class Pug can directly access the members of the class Animal and Dog.

Hierarchical Inheritance in C++

In hierarchical inheritance, multiple child classes can inherit the property from a single-parent class.



As you can see from the image above, the child classes B, C, and D are derived from the single base class A.

Let's see the syntax to perform hierarchical inheritance in cpp:

```
class A
{


};
class B: visibility_mode A
{


};
class C: visibility_mode A
{


};
class D: visibility_mode A
{
```

```
};
```

Here, class A is the parent class, and classes B, C, and D are the child classes.
Let's see the example of hierarchical inheritance:

```
// base class

class Animal

{

public:

   // constructor of the base class

   Animal()

   {

      cout << "I am an animal.\n\n";

   }

};

 // derived class

class Dog: public Animal
{

};
 // derived class

class Pig: public Animal
{

};
 // derived class

class Cat: public Animal
```

```
{

};
int main()
{
   // create object of the child class
   Dog obj1;
   Pig pbj2;
   Cat obj3;
}
```

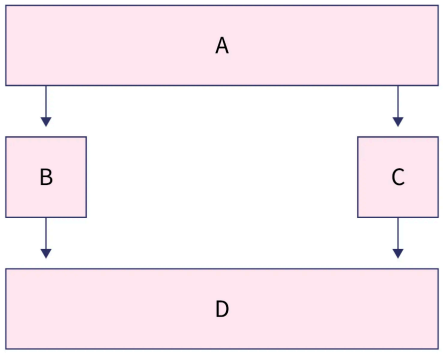
The output of the above example is:

```
I am an animal.
I am an animal.
I am an animal.
```

Here, in the example, all the derived classes can access the base class Animal public members. When it creates objects of these three derived classes, it calls the base class's constructor for all three objects.

Hybrid Inheritance in C++

As the name hybrid suggests, it combines two or more types of inheritances. Due to the mixture of more types of inheritance, it is one of the most complexes.



SCALER Topics

As you can see from the above image, hybrid inheritance is the combination of hierarchical and multiple inheritances.

Let's see the syntax to perform hierarchical inheritance in cpp:

```
class A
{


};
class B: visibility_mode A
{


};
class C: visibility_mode A
{


};
class D: visibility_mode B, visibility_mode C
{


};
```

Here, class A is the parent class, classes B, and C are the child classes, and classes B and C are the parent class for class D, which inherits its properties from classes B and C.

Let's see the example of hybrid inheritance:

```
// base class

class Animal

{

public:

   // constructor of the base class

   Animal()

   {

      cout << "I am an animal.\n\n";

   }

};

 // derived class

class Dog: public Animal
{

};
 // derived class

class Pig: public Animal
{

};
 // derived class

class Cute: public Dog, public Pig
{
```

```
};
int main()
{
   // create an object of the child's class
   Cute obj;
}
```

The output of the above example is:

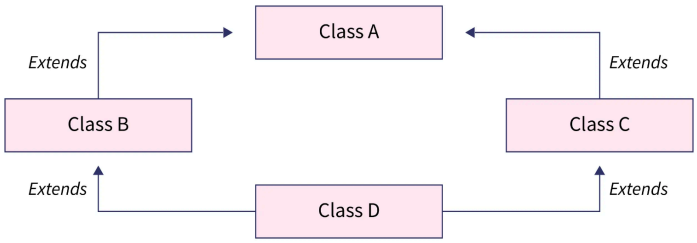
```
I am an animal.
I am an animal.
```

Here, in the example, the constructors of all its superclasses are called when the object of the derived class Cute is created.

Ambiguity in inheritance / Diamond Problem

The diamond problem is a problem or an ambiguity that arises because of multiple inheritances where there are a lot of chances of having multiple properties or methods with the same name in different sub-classes, which may lead to ambiguity.

The diamond problem occurs when two classes B and C inherit from class A, and then class D inherits from both B and C, so if there is a method defined in class A that class B and C have overridden, then class D faces the ambiguity of which version of the method does class D inherit: that of B, or that of C?



This problem is known as the diamond problem because of the shape of the class inheritance diagram in this situation, as you can see from the image above.

The following example below will help to illustrate the ambiguous situation caused because of multiple inheritances:

```
// base class
```

```
class A

{

public:

   int var1;

};

// class 1

class B : pblic A

{

public:

   int var2;

};

// class 2

class C: public A

{

public:

   int var3;

};

// derived class 3

class D : public B, public C

{
```

```
public:

    int var4;

};

int main()

{

    // create an object of the derived_class

    D obj;

    obj.var1 = 10;  // ambiguous

    obj.var2 = 20;

    obj.var3 = 30;

    obj.var4 = obj.var1 + obj.var2 + obj.var3;

    cout << obj.var4 << endl;
}
```

In the above example, the error will be thrown, stating that var1 is ambiguous and the ambiguity is being caused. This happens because there are two copies of the data member var1 of the base class A, one for class B and one for class C. When the object obj of the child class D tries to access this member variable, it is not specified which copy of the variable var1 is to be used. Hence the ambiguity caused by an error is thrown.

There are two ways to deal with the diamond problem and to remove the ambiguity:

- Using the Scope Resolution Operator:

Let's consider the same example above and see how the scope resolution operator can help deal with the diamond problem.

```
// base class

class A

{
```

```
public:
    int var1;
};
// class 1
clas B: public A
{
public:
    int var2;
};
// class 2
class C: public A
{
public:
    int var3;
};
// derived class 3
class D : public B, public C
{
public:
    int var4;
```

```
};

int main()

{

   // create an object of the derived_class

   D obj;

   obj.A::var1 = 10;  // now it is unambiguous

   obj.var2 = 20;

   obj.var3 = 30;

   obj.var4 = obj.A::var1 + obj.var2 + obj.var3;

   cout << obj.var4 << endl;
}
```

The output of the above program will be:

```
60
```

The ambiguity error will be removed as the scope resolution operator specifies which copy of var1 will be used.

```
obj.A::var1;
```

Here, the A's version of var1 is accessed.

- Using Virtual Keyword:

With the help of the scope resolution operator, we were able to remove the error of ambiguity, but still, there are two copies of the parent class. If you require only one copy of the base class, there is another solution using the virtual keyword.

The virtual keyword allows only one copy of the parent class to be created, and then we can call the object of the child class in a way we normally do, and using that object, we can access the data members of the parent class in the usual way.

The example below will show the use of the virtual keyword to remove ambiguity in the diamond problem.

```
// base class

class A

{

public:

   int var1;

};

class B: virtual public A

{

public:

   int var2;

};

class C: virtual public A

{

public:

   int var3;

};

class D : public B, public C

{

public:

   int var4;
```

```
};

int main()

{

   // create an object of the derived_class

   D obj;

   obj.var1 = 10;  // it is now unambiguous, and we can access it in a usual way

   obj.var2 = 20;

   obj.var3 = 30;

   obj.var4 = obj.var1 + obj.var2 + obj.var3;

   cout << obj.var4 << endl;

}
```

The output of the above program will be:
60

In the above example, classes B and C inherit the base class A as virtual, so it will only create a single copy of the base class A. So, now the derived class D object has only one copy of the base class data members. Hence only one copy of the variable var1 will be created, which makes it error-free and unambiguous.

Function Overriding

- **Definition:**

Function Overriding in C++ occurs when a derived class defines a function with the **same name, return type, and parameters** as a function in its base class. The function in the derived class **overrides** the version in the base class.

- **Need:**
  - To implement **runtime polymorphism**.
  - To allow derived classes to **customize or extend** the behavior of base class methods.

- Used in **dynamic binding** with virtual functions.

🔹 Use Cases:

- A base class "Shape" has a `draw()` method, and derived classes like "Circle" or "Rectangle" override it for specific drawing behavior.
- Employee class has `calculateSalary()`, overridden differently for Manager and Clerk.

🔹 Syntax:

cpp
CopyEdit

```
class Base {
public:
    virtual void display() {
        cout << "Base Display" << endl;
    }
};

class Derived : public Base {
public:
    void display() override { // overrides base function
        cout << "Derived Display" << endl;
    }
};
```

🔹 Example 1 (Beginner):

cpp
CopyEdit

```
#include<iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal makes sound\n";
    }
};

class Dog : public Animal {
public:
    void sound() override {
```

```
        cout << "Dog barks\n";
    }
};

int main() {
    Animal* a;
    Dog d;
    a = &d;
    a->sound();  // Output: Dog barks
}
```

🔹 **Example 2 (Beginner):**

cpp
CopyEdit

```
class Vehicle {
public:
    virtual void start() {
        cout << "Vehicle starts\n";
    }
};

class Bike : public Vehicle {
public:
    void start() override {
        cout << "Bike starts with kick\n";
    }
};

int main() {
    Bike b;
    b.start(); // Output: Bike starts with kick
}
```

🔹 **Example 3 (Medium):**

cpp
CopyEdit

```
class Account {
public:
    virtual void calculateInterest() {
```

```
        cout << "Base interest\n";
    }
};

class Savings : public Account {
public:
    void calculateInterest() override {
        cout << "Savings interest: 4%\n";
    }
};

class FixedDeposit : public Account {
public:
    void calculateInterest() override {
        cout << "Fixed Deposit interest: 7%\n";
    }
};

int main() {
    Account* acc;
    Savings s;
    FixedDeposit f;

    acc = &s;
    acc->calculateInterest();  // Output: Savings interest: 4%

    acc = &f;
    acc->calculateInterest();  // Output: Fixed Deposit interest: 7%
}
```

Nesting of Classes

🔹 Definition:

Nesting of classes refers to the concept of **defining a class within another class**. The inner class is known as the nested class, and it can be **public, private, or protected**.

🔹 Need:

- To logically group classes that are **only used in one place**.
- To **encapsulate helper classes** inside a primary class.

- Enhances **readability and scope restriction**.

🔹 Use Cases:

- A class `University` containing a nested class `Department`.
- An `ATM` system with nested `CardReader`.

🔹 Syntax:

cpp
CopyEdit

```
class Outer {
public:
    class Inner {
    public:
        void show() {
            cout << "Inner class method\n";
        }
    };
};
```

🔹 Example 1 (Beginner):

cpp
CopyEdit

```
class Library {
public:
    class Book {
    public:
        void display() {
            cout << "Book is available\n";
        }
    };
};

int main() {
    Library::Book b;
    b.display();
}
```

🔹 Example 2 (Beginner):

cpp

CopyEdit

```
class Engine {
public:
    class SparkPlug {
    public:
        void ignite() {
            cout << "Spark Plug ignited\n";
        }
    };
};

int main() {
    Engine::SparkPlug sp;
    sp.ignite();
}
```

🔹 **Example 3 (Medium):**

cpp

CopyEdit

```
class Company {
    static int companyCode;
public:
    class Employee {
    public:
        void showCode() {
            cout << "Company Code: " << Company::companyCode << endl;
        }
    };
};

int Company::companyCode = 9876;

int main() {
    Company::Employee e;
    e.showCode();
}
```

Inheritance of Nested Classes

🔹 Definition:

A nested class (inner class) can also be **inherited** just like regular classes. This is done by creating a derived class from the nested class using the scope resolution of the enclosing class.

🔹 Need:

- When you want to **extend a specific part** of an outer class's structure.
- Useful when the nested class serves as a base template for specific variations.

🔹 Use Cases:

- `University::Department` → inherited by `EngineeringDepartment`.
- `Company::Employee` → inherited by `Manager`.

🔹 Syntax:

cpp
CopyEdit

```
class Outer {
public:
    class Inner {
    public:
        void show() {
            cout << "Inner base\n";
        }
    };
};

class Derived : public Outer::Inner {
public:
    void display() {
        cout << "Derived from nested class\n";
    }
};
```

🔹 Example 1 (Beginner):

cpp
CopyEdit

```
class School {
public:
    class Teacher {
    public:
        void teach() {
```

```
            cout << "Teaching students\n";
        }
    };
};

class MathsTeacher : public School::Teacher {
public:
    void subject() {
        cout << "Subject: Mathematics\n";
    }
};

int main() {
    MathsTeacher mt;
    mt.teach();
    mt.subject();
}
```

🔹 **Example 2 (Beginner):**

cpp
CopyEdit

```
class Vehicle {
public:
    class Engine {
    public:
        void start() {
            cout << "Engine starts\n";
        }
    };
};

class SportsEngine : public Vehicle::Engine {
public:
    void boost() {
        cout << "Boost mode on\n";
    }
};

int main() {
    SportsEngine se;
```

```
    se.start();
    se.boost();
}
```

🔹 **Example 3 (Medium):**

cpp
CopyEdit

```
class Computer {
public:
    class CPU {
    public:
        void process() {
            cout << "Processing data\n";
        }
    };
};

class AdvancedCPU : public Computer::CPU {
public:
    void optimize() {
        cout << "Optimizing instructions\n";
    }
};

int main() {
    AdvancedCPU ac;
    ac.process();
    ac.optimize();
}
```